

Интерфейс

– это контракт (набор методов). Если тип реализует все методы интерфейса, он автоматически удовлетворяет этому интерфейсу

Особенность: занимает 16 байт на 64б арх

eface – пустой интерфейс (interface{}, он же any)

```
type eface struct {
    _type *_type // информация о типе
    data unsafe.Pointer // указатель на данные
}

type _type struct {
    size      uintptr // размер типа в байтах
    ptrdata   uintptr // размер части типа, содержащей указатели
    hash      uint32  // уникальный хеш типа, помогает при сравнении
    tflag     tflag   // флаги дополнительной информации о типе
    align     uint8   // выравнивание для переменной этого типа
    fieldalign uint8   // выравнивание для поля структуры этого типа
    kind      uint8   // базовый вид типа (int, string, ptr, struct и т.д.)
    equal     func(unsafe.Pointer, unsafe.Pointer) bool // функция для сравнения
    // двух значений типа
    gcdata    *byte   // информация для сборщика мусора (битовая карта)
    str       nameOff // смещение на строку с именем типа
    ptrToThis typeOff // смещение на тип, являющийся указателем на этот
}

iface – интерфейс с методами (io.Reader, error и т.д.)

type iface struct {
    tab *itab // указатель на таблицу методов + информация о типе
    data unsafe.Pointer // указатель на данные
}

type itab struct {
    inter *interfacetype // информация об интерфейсе
    _type *_type // информация о конкретном типе
    hash uint32 // копия _type.hash для быстрого сравнения
    fun [1]uintptr // массив указателей на методы (variable size)
}
```

Ключевой момент:

В обоих случаях интерфейс – это структура из двух полей. И в обоих случаях интерфейс считается nil только когда оба поля равны nil.

Природа nil для интерфейсов и типизированный nil

nil:

- nil-указатель – когда tab != nil, data == nil
- Пустой интерфейс (nil-интерфейс) – когда tab == nil, data == nil

Когда мы присваиваем nil-указатель интерфейсу, мы создаем типизированный nil:

```
var p *int = nil // p – nil-указатель на int
var i interface{} = p // i – это eface{_type: *int, data: nil}
```

В этот момент:

- _type указывает на информацию о типе *int (не nil)

- data указывает на nil (пустой указатель)

Поэтому `i == nil` вернет `false`, потому что `_type != nil`

```
fmt.Println(i == nil) // false
```

Сравнение разных вариантов nil

```
func main() {
    // 1. Пустой интерфейс без типа
    var i1 interface{} // eface{_type: nil, data: nil}
    fmt.Println(i1 == nil) // true

    // 2. Типизированный nil (самый частый баг!)
    var p *int = nil
    var i2 interface{} = p // eface{_type: *int, data: nil}
    fmt.Println(i2 == nil) // false

    // 3. Интерфейс с методами и nil-указателем
    var r io.Reader
    var b *bytes.Buffer = nil
    r = b // iface{tab: *Buffer, data: nil}
    fmt.Println(r == nil) // false

    // 4. Интерфейс с методами без типа
    var r2 io.Reader // iface{tab: nil, data: nil}
    fmt.Println(r2 == nil) // true
}
```

Популярное заблуждение

```
type MyError struct {
    Msg string
}

func (e *MyError) Error() string {
    return e.Msg
}

func DoSomething() error {
    var err *MyError = nil // nil-указатель на MyError
    return err // возвращает iface{tab: *MyError, data: nil}
}

func main() {
    err := DoSomething()
    if err != nil { // true! (потому что tab != nil)
        fmt.Println("Ошибка:", err) // выведет "Ошибка: "
    }
}
```

Когда мы возвращаем nil-указатель на MyError, мы возвращаем интерфейс с заполненным полем tab.

Решение: Всегда возвращайте литерал nil:

```
func DoSomething() error {
    if something {
        return &MyError{Msg: "oops"}
    }
}
```

```
    return nil // правильный nil-интерфейс
}
```

Как правильно проверить интерфейс на nil?

Вариант 1. reflect

```
func IsNil(i interface{}) bool {
    if i == nil {
        return true
    }
    v := reflect.ValueOf(i)
    switch v.Kind() {
    case reflect.Ptr, reflect.Interface, reflect.Map, reflect.Slice, reflect.Chan,
reflect.Func:
        return v.IsNil()
    default:
        return false
    }
}
```

Вариант 2. type assertion (если знаем тип)

```
func IsNilError(err error) bool {
    // error – это интерфейс, используем reflect
    return err == nil || reflect.ValueOf(err).IsNil()
}
```

Почему any быстрее, чем интерфейс с методами?

```
// eface -- быстрее
var a any = 42 // просто _type + data

// iface -- чуть медленнее
var r io.Reader = strings.NewReader("hello") // нужно создать itab
```

Как не стрелять себе в ногу

Правило 1: Возвращайте конкретные типы, а не интерфейсы

```
// Плохо
func NewReader() io.Reader {
    return nil // опасно!
}

// Хорошо
func NewReader() *MyReader {
    return nil // безопасно, если проверять *MyReader == nil
}

func GetUser(id int) (User, error) {
    if id <= 0 {
        return User{}, nil // а не var err *MyError = nil
    }
    return User{ID: id}, nil
}
```

Исключение:

для error – наоборот, возвращаем интерфейс error, но не через промежуточную typed-nil переменную

Правило 2: Проверяйте на nil до присваивания интерфейсу

```
// Плохо
var result *Result
var i interface{} = result
if i != nil { ... } // никогда не сработает
```

```
// Хорошо
var result *Result
if result != nil {
    var i interface{} = result
    // работаем с i
}
```